

A Simple LLM, Start to Finish

Contents

A Simple LLM, Start to Finish: Architecture, Fully Expanded Worked Example, and Training Signal	1
Part 1 — The Architecture	1
Part 2 — Full Worked Example: “<bos> I love AI” Through One Decoder Layer	3
Appendix A — Fully Expanded Per-Token Arithmetic, All 16 Steps, in LaTeX Matrix Form	4

A Simple LLM, Start to Finish: Architecture, Fully Expanded Worked Example, and Training Signal

A complete reference for the “vanilla” decoder-only Transformer — the common ancestor that both BERT (bidirectional, covered in the earlier BERT document) and DeepSeek-R1 (MLA + RoPE + MoE + MTP + FP8, covered in the earlier DeepSeek document) are variations of. This document deliberately uses the *plainest* version of every choice — standard multi-head attention with no compression, standard learned positional embeddings with no rotation, a single dense feed-forward block with no experts, and a single next-token prediction with no lookahead — so that every design choice in the more advanced documents can be understood as “what changed, and why,” relative to this baseline. It closes with the one piece neither earlier document showed: the actual training loss.

Part 1 — The Architecture

What this model is

A decoder-only, autoregressive language model — the same family as GPT-2, GPT-3, and early Llama. “Autoregressive” means it generates one token at a time, always conditioning only on what came before, and “decoder-only” means there’s a single stack of causally-masked Transformer blocks (no separate encoder, unlike a translation model). This is the architecture that both BERT and DeepSeek-R1 depart from in different directions: BERT removes the causal mask (bidirectional) and changes the training objective (masked-token prediction instead of next-token prediction); DeepSeek-R1 keeps the causal, next-token-prediction core but replaces individual components (MLA instead of standard attention, MoE instead of a dense FFN, and so on).

Tokenization and embedding

Real tokenizers (GPT-2 uses a BPE — Byte-Pair Encoding — tokenizer, ~50K vocabulary) build a fixed vocabulary of subword units from corpus statistics, then map text to a sequence of integer IDs by greedily matching the longest known piece at each position. The **token embedding matrix** is a single learned table of shape $[\text{vocab_size}, d_{\text{model}}]$; looking up a token’s embedding is a simple row index, no computation involved.

Positional embedding — the simplest version

Unlike DeepSeek’s RoPE (a rotation applied inside attention) or BERT’s sinusoidal formula, the simplest version of positional encoding is a **second learned lookup table**, indexed purely by position (0, 1, 2, 3, ...), completely independent of which token happens to sit there. The position-0 vector is the same regardless of whether the token at position 0 is “I” or “The”; it is *added* directly to the token embedding:

$$X = \text{TokenEmbedding}(\text{token_id}) + \text{PositionEmbedding}(\text{position})$$

This is exactly what GPT-2 does. Its main limitation (and the reason RoPE became popular later) is that it has no learned representation for any position beyond the maximum sequence length seen during training — a rotation-based scheme like RoPE degrades more gracefully at unseen lengths.

Pre-norm, standard LayerNorm

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \text{mean}(x)}{\sqrt{\text{var}(x) + \epsilon}} + \beta$$

Unlike DeepSeek’s RMSNorm (which skips mean-centering for speed), standard LayerNorm centers the vector at zero and rescales it to unit variance before applying a learned scale (γ) and shift (β). “Pre-norm” means this normalization is applied to the input of each sub-block (attention, then feed-forward) rather than to its output — GPT-2 popularized this placement because it makes very deep stacks (dozens of layers) noticeably easier to train than the original Transformer’s post-norm placement (which BERT uses).

Standard multi-head self-attention — no compression, no rotation

$$Q = XW_q \quad K = XW_k \quad V = XW_v$$
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{causal_mask} \right) V$$

Each head projects the input directly into its own Query, Key, and Value using its own full-size weight matrices — there is no shared latent bottleneck (contrast with MLA’s down-projection into a compressed cache), and position was already baked into X at the embedding step, so nothing needs to be rotated here (contrast with RoPE). The **causal mask** sets any attention score from position i to a later position j to negative infinity before softmax, guaranteeing token i can never see the future — this is what makes the model usable for autoregressive generation one token at a time.

Standard dense feed-forward network — no experts

$$\text{FFN}(x) = \text{GELU}(xW_1)W_2$$

A single feed-forward block, applied identically to every token, with no routing and no per-token specialization (contrast with DeepSeekMoE’s shared+routed experts). This is the computationally simplest choice: every token costs the same amount of compute, with none of MoE’s added complexity around load balancing or expert selection.

Output layer and next-token prediction

The final hidden state at each position is compared — via dot product — against every word’s own embedding vector (frequently the *same* embedding matrix used at the input, a technique called weight tying), producing one raw “logit” per vocabulary word. Softmax turns these logits into a genuine probability distribution:

$$P(\text{word}) = \frac{e^{\text{logit}(\text{word}) - \max}}{\sum_w e^{\text{logit}(w) - \max}}$$

Whichever word has the highest probability is the model’s single-token prediction (contrast with DeepSeek’s Multi-Token Prediction, which adds extra lightweight modules to also predict 2+ tokens ahead during training).

The training signal — cross-entropy loss

Neither the BERT document nor the DeepSeek document showed this step, and it’s the one that actually makes training possible:

$$\mathcal{L} = -\log(P(\text{true_next_token}))$$

At every position, the model’s predicted probability for the token that *actually* came next in the real training text is extracted, and its negative log is the loss for that position. A perfect prediction ($P = 1.0$) gives a loss of $-\log(1) = 0$; a confident wrong prediction gives a very large loss. Averaged across all positions in a batch, this single scalar is what gradient descent minimizes — its gradient flows backward through literally every weight matrix used in the forward pass (the output projection, the FFN, every attention weight matrix, all the way to the embedding tables), nudging each one a tiny amount in the direction that would have made the true token more likely. Repeating this over billions of tokens of real text is the entirety of how a language model learns.

Typical real-world scale

	GPT-2 (small)	GPT-2 (XL)	GPT-3
Layers	12	48	96
Hidden size (d_{model})	768	1600	12288
Attention heads	12	25	96
Parameters	117M	1.5B	175B

Every one of these layers repeats the exact sequence covered in Part 2 below (LayerNorm $\rightarrow$ attention $\rightarrow$ residual $\rightarrow$ LayerNorm $\rightarrow$ FFN $\rightarrow$ residual), stacked back to back, each layer’s output feeding the next layer’s input.

Part 2 — Full Worked Example: “<bos> I love AI” Through One Decoder Layer

Sequence: “<bos> I love AI” (4 tokens, the same sentence used in the DeepSeek document, so the two can be compared step for step). Toy dims: $d_{model} = 4$, 2 heads, $d_k = 2$ per head. This section walks the narrative once at a readable pace; **Appendix A** repeats every step with the arithmetic fully expanded in LaTeX matrix form for all 4 tokens.

Steps 2-4: Tokenize, embed, add position

Look up each token’s row in the embedding table, look up each *position’s* row in a separate positional embedding table, and sum them elementwise. Unlike DeepSeek, nothing here is compressed or rotated — position is simply added once, up front, and every downstream step just sees X as an ordinary vector with no special structure.

Step 5: Pre-attention LayerNorm

Standard, mean-centered LayerNorm, applied to X before it reaches attention (pre-norm placement, same choice DeepSeek makes, but with LayerNorm’s mean-centering instead of RMSNorm’s).

Steps 6-7: Standard multi-head attention with a causal mask

Each head projects the normalized input directly into Q, K, V using its own full-size weight matrix — no latent bottleneck, no rotation. Scaled dot-product attention proceeds exactly as in the DeepSeek document, with the same causal mask blocking any token from attending to a later position, producing the same strict “staircase” pattern in the softmax weights.

Steps 8-10: Concat heads, project, residual, second LayerNorm

Concatenate every head's output, project through W_o , add back to the *original* (pre-normalization) residual stream, then apply a second LayerNorm before the feed-forward block.

Steps 11-14: Standard dense feed-forward network, residual, final LayerNorm

A single FFN — $\text{GELU}(xW_1)W_2$ — runs identically for every token; there is no routing decision and no expert selection anywhere in this step, in contrast to DeepSeek's MoE block. Add the residual, apply the final LayerNorm, and this is the output of one full decoder layer. In a real model, this entire sequence (Steps 5-14) repeats for every layer in the stack — 12, 48, or 96 times depending on model size — each layer's output feeding the next layer's input.

Step 15: Next-token prediction — how a hidden vector becomes a word

Every position's final hidden state is compared (via dot product against the tied embedding matrix) to every candidate word, producing one logit per vocabulary word, then softmax turns those logits into a genuine probability distribution. This is the exact mechanism covered in detail in the DeepSeek document's Step 18 — repeated here in full in Appendix A, since it's just as central to this simpler model.

Step 16: Cross-entropy loss — the step neither earlier document showed

Compare the predicted probability distribution at each position against the token that *actually* came next in real training text, take the negative log of the probability assigned to that correct token, and average across positions. This scalar is the actual quantity that gradient descent minimizes, and its gradient is what propagates backward through every weight matrix in this document during real training.

Appendix A — Fully Expanded Per-Token Arithmetic, All 16 Steps, in LaTeX Matrix Form

Every weight matrix is shown in full before it's used, every input as a full matrix, and every step ends with the resulting output matrix.

Step 2: Token embeddings (looked up from the vocabulary table)

$\mathbf{X}_{\text{token}}$ (shape 4×4):

<bos>	0.0500	0.0500	0.0500	0.0500
I	0.2000	0.1000	-0.1000	0.0000
love	-0.1000	0.3000	0.2000	0.1000
AI	0.3000	-0.2000	0.1000	0.2000

Step 3: Learned positional embeddings (a SEPARATE lookup table, indexed by position)

Positional embedding table (shape 4×4):

pos 0	0.0000	0.1000	0.0000	0.0000
pos 1	0.1000	0.0000	0.1000	0.0500
pos 2	0.0000	0.1000	-0.0500	0.1000
pos 3	0.0500	-0.0500	0.1000	0.0000

Step 4: Input $\mathbf{X}$ = Token embedding + Positional embedding, every token

$$\text{<bos>} : [0.0500, 0.0500, 0.0500, 0.0500] + [0.0000, 0.1000, 0.0000, 0.0000] = [0.0500, 0.1500, 0.0500, 0.0500]$$

$$\text{I} : [0.2000, 0.1000, -0.1000, 0.0000] + [0.1000, 0.0000, 0.1000, 0.0500] = [0.3000, 0.1000, 0.0000, 0.0500]$$

$$\text{love} : [-0.1000, 0.3000, 0.2000, 0.1000] + [0.0000, 0.1000, -0.0500, 0.1000] = [-0.1000, 0.4000, 0.1500, 0.2000]$$

$$\text{AI} : [0.3000, -0.2000, 0.1000, 0.2000] + [0.0500, -0.0500, 0.1000, 0.0000] = [0.3500, -0.2500, 0.2000, 0.2000]$$

Resulting X matrix (shape 4×4):

$$\begin{array}{c|cccc} <\text{bos}> & 0.0500 & 0.1500 & 0.0500 & 0.0500 \\ \text{I} & 0.3000 & 0.1000 & 0.0000 & 0.0500 \\ \text{love} & -0.1000 & 0.4000 & 0.1500 & 0.2000 \\ \text{AI} & 0.3500 & -0.2500 & 0.2000 & 0.2000 \end{array}$$

Step 5: Pre-attention LayerNorm

Input matrix (every token as a row) (shape 4×4):

$$\begin{array}{c|cccc} <\text{bos}> & 0.0500 & 0.1500 & 0.0500 & 0.0500 \\ \text{I} & 0.3000 & 0.1000 & 0.0000 & 0.0500 \\ \text{love} & -0.1000 & 0.4000 & 0.1500 & 0.2000 \\ \text{AI} & 0.3500 & -0.2500 & 0.2000 & 0.2000 \end{array}$$

$\gamma = 1, \beta = 0$ (identity affine, to isolate the normalization math)

$$<\text{bos}>: v = [0.0500, 0.1500, 0.0500, 0.0500]$$

$$\text{mean} = \frac{0.0500 + 0.1500 + 0.0500 + 0.0500}{4} = 0.0750$$

$$\text{deviations} = [-0.0250, 0.0750, -0.0250, -0.0250] \Rightarrow \text{variance} = \frac{0.0075}{4} = 0.0019$$

$$\text{std} = \sqrt{0.0019 + 10^{-5}} = 0.0434$$

$$\text{normalized} = \left[\frac{-0.0250}{0.0434} = -0.5758, \frac{0.0750}{0.0434} = 1.7275, \frac{-0.0250}{0.0434} = -0.5758, \frac{-0.0250}{0.0434} = -0.5758 \right]$$

$$\text{I}: v = [0.3000, 0.1000, 0.0000, 0.0500]$$

$$\text{mean} = \frac{0.3000 + 0.1000 + 0.0000 + 0.0500}{4} = 0.1125$$

$$\text{deviations} = [0.1875, -0.0125, -0.1125, -0.0625] \Rightarrow \text{variance} = \frac{0.0519}{4} = 0.0130$$

$$\text{std} = \sqrt{0.0130 + 10^{-5}} = 0.1139$$

$$\text{normalized} = \left[\frac{0.1875}{0.1139} = 1.6458, \frac{-0.0125}{0.1139} = -0.1097, \frac{-0.1125}{0.1139} = -0.9875, \frac{-0.0625}{0.1139} = -0.5486 \right]$$

$$\text{love}: v = [-0.1000, 0.4000, 0.1500, 0.2000]$$

$$\text{mean} = \frac{-0.1000 + 0.4000 + 0.1500 + 0.2000}{4} = 0.1625$$

$$\text{deviations} = [-0.2625, 0.2375, -0.0125, 0.0375] \Rightarrow \text{variance} = \frac{0.1269}{4} = 0.0317$$

$$\text{std} = \sqrt{0.0317 + 10^{-5}} = 0.1781$$

$$\text{normalized} = \left[\frac{-0.2625}{0.1781} = -1.4737, \frac{0.2375}{0.1781} = 1.3333, \frac{-0.0125}{0.1781} = -0.0702, \frac{0.0375}{0.1781} = 0.2105 \right]$$

AI: $v = [0.3500, -0.2500, 0.2000, 0.2000]$

$$\text{mean} = \frac{0.3500 + -0.2500 + 0.2000 + 0.2000}{4} = 0.1250$$

$$\text{deviations} = [0.2250, -0.3750, 0.0750, 0.0750] \Rightarrow \text{variance} = \frac{0.2025}{4} = 0.0506$$

$$\text{std} = \sqrt{0.0506 + 10^{-5}} = 0.2250$$

$$\text{normalized} = \left[\frac{0.2250}{0.2250} = 0.9999, \frac{-0.3750}{0.2250} = -1.6665, \frac{0.0750}{0.2250} = 0.3333, \frac{0.0750}{0.2250} = 0.3333 \right]$$

Resulting normalized matrix (shape 4×4):

$\langle \text{bos} \rangle$	-0.5758	1.7275	-0.5758	-0.5758
I	1.6458	-0.1097	-0.9875	-0.5486
love	-1.4737	1.3333	-0.0702	0.2105
AI	0.9999	-1.6665	0.3333	0.3333

Step 6 (Head 1): standard Q, K, V projections (direct, NO compression)

W_{q_1} (shape 4×2):

$$\begin{bmatrix} 1.0000 & 0.0000 \\ 0.0000 & 1.0000 \\ 0.5000 & 0.0000 \\ 0.0000 & 0.5000 \end{bmatrix}$$

X_n (**every token as a row**) (shape 4×4):

$\langle \text{bos} \rangle$	-0.5758	1.7275	-0.5758	-0.5758
I	1.6458	-0.1097	-0.9875	-0.5486
love	-1.4737	1.3333	-0.0702	0.2105
AI	0.9999	-1.6665	0.3333	0.3333

Expanded arithmetic, $Q = X_n \cdot W_{q_1}$, every token:

$\langle \text{bos} \rangle$ (X_n row = $[-0.5758, 1.7275, -0.5758, -0.5758]$)

$$Q[0] = (-0.5758 \times 1.0000) + (1.7275 \times 0.0000) + (-0.5758 \times 0.5000) + (-0.5758 \times 0.0000) = -0.8637$$

$$Q[1] = (-0.5758 \times 0.0000) + (1.7275 \times 1.0000) + (-0.5758 \times 0.0000) + (-0.5758 \times 0.5000) = 1.4395$$

I (X_n row = $[1.6458, -0.1097, -0.9875, -0.5486]$)

$$Q[0] = (1.6458 \times 1.0000) + (-0.1097 \times 0.0000) + (-0.9875 \times 0.5000) + (-0.5486 \times 0.0000) = 1.1521$$

$$Q[1] = (1.6458 \times 0.0000) + (-0.1097 \times 1.0000) + (-0.9875 \times 0.0000) + (-0.5486 \times 0.5000) = -0.3840$$

love (X_n row = $[-1.4737, 1.3333, -0.0702, 0.2105]$)

$$Q[0] = (-1.4737 \times 1.0000) + (1.3333 \times 0.0000) + (-0.0702 \times 0.5000) + (0.2105 \times 0.0000) = -1.5088$$

$$Q[1] = (-1.4737 \times 0.0000) + (1.3333 \times 1.0000) + (-0.0702 \times 0.0000) + (0.2105 \times 0.5000) = 1.4386$$

$$AI \text{ (} Xn \text{ row} = [0.9999, -1.6665, 0.3333, 0.3333])$$

$$Q[0] = (0.9999 \times 1.0000) + (-1.6665 \times 0.0000) + (0.3333 \times 0.5000) + (0.3333 \times 0.0000) = 1.1666$$

$$Q[1] = (0.9999 \times 0.0000) + (-1.6665 \times 1.0000) + (0.3333 \times 0.0000) + (0.3333 \times 0.5000) = -1.4999$$

Resulting Q matrix (shape 4×2):

$$\begin{array}{c|cc} <bos> & -0.8637 & 1.4395 \\ I & 1.1521 & -0.3840 \\ love & -1.5088 & 1.4386 \\ AI & 1.1666 & -1.4999 \end{array}$$

Wk_1 (shape 4×2):

$$\begin{bmatrix} 0.8000 & 0.0000 \\ 0.0000 & 0.8000 \\ 0.3000 & 0.0000 \\ 0.0000 & 0.3000 \end{bmatrix}$$

Xn (**every token as a row**) (shape 4×4):

$$\begin{array}{c|cccc} <bos> & -0.5758 & 1.7275 & -0.5758 & -0.5758 \\ I & 1.6458 & -0.1097 & -0.9875 & -0.5486 \\ love & -1.4737 & 1.3333 & -0.0702 & 0.2105 \\ AI & 0.9999 & -1.6665 & 0.3333 & 0.3333 \end{array}$$

Expanded arithmetic, $K = Xn \cdot Wk_1$, every token:

$$<bos> \text{ (} Xn \text{ row} = [-0.5758, 1.7275, -0.5758, -0.5758])$$

$$K[0] = (-0.5758 \times 0.8000) + (1.7275 \times 0.0000) + (-0.5758 \times 0.3000) + (-0.5758 \times 0.0000) = -0.6334$$

$$K[1] = (-0.5758 \times 0.0000) + (1.7275 \times 0.8000) + (-0.5758 \times 0.0000) + (-0.5758 \times 0.3000) = 1.2092$$

$$I \text{ (} Xn \text{ row} = [1.6458, -0.1097, -0.9875, -0.5486])$$

$$K[0] = (1.6458 \times 0.8000) + (-0.1097 \times 0.0000) + (-0.9875 \times 0.3000) + (-0.5486 \times 0.0000) = 1.0204$$

$$K[1] = (1.6458 \times 0.0000) + (-0.1097 \times 0.8000) + (-0.9875 \times 0.0000) + (-0.5486 \times 0.3000) = -0.2524$$

$$love \text{ (} Xn \text{ row} = [-1.4737, 1.3333, -0.0702, 0.2105])$$

$$K[0] = (-1.4737 \times 0.8000) + (1.3333 \times 0.0000) + (-0.0702 \times 0.3000) + (0.2105 \times 0.0000) = -1.2000$$

$$K[1] = (-1.4737 \times 0.0000) + (1.3333 \times 0.8000) + (-0.0702 \times 0.0000) + (0.2105 \times 0.3000) = 1.1298$$

$$AI \text{ (} Xn \text{ row} = [0.9999, -1.6665, 0.3333, 0.3333])$$

$$K[0] = (0.9999 \times 0.8000) + (-1.6665 \times 0.0000) + (0.3333 \times 0.3000) + (0.3333 \times 0.0000) = 0.8999$$

$$K[1] = (0.9999 \times 0.0000) + (-1.6665 \times 0.8000) + (0.3333 \times 0.0000) + (0.3333 \times 0.3000) = -1.2332$$

Resulting K matrix (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & -0.6334 & 1.2092 \\ \text{I} & 1.0204 & -0.2524 \\ \text{love} & -1.2000 & 1.1298 \\ \text{AI} & 0.8999 & -1.2332 \end{array}$$

Wv_1 (shape 4×2):

$$\begin{bmatrix} 0.2000 & 0.1000 \\ 0.1000 & 0.2000 \\ 0.0000 & 0.3000 \\ 0.3000 & 0.0000 \end{bmatrix}$$

Xn (**every token as a row**) (shape 4×4):

$$\begin{array}{c|cccc} \langle \text{bos} \rangle & -0.5758 & 1.7275 & -0.5758 & -0.5758 \\ \text{I} & 1.6458 & -0.1097 & -0.9875 & -0.5486 \\ \text{love} & -1.4737 & 1.3333 & -0.0702 & 0.2105 \\ \text{AI} & 0.9999 & -1.6665 & 0.3333 & 0.3333 \end{array}$$

Expanded arithmetic, $V = Xn \cdot Wv_1$, every token:

$\langle \text{bos} \rangle$ (Xn row = $[-0.5758, 1.7275, -0.5758, -0.5758]$)

$$V[0] = (-0.5758 \times 0.2000) + (1.7275 \times 0.1000) + (-0.5758 \times 0.0000) + (-0.5758 \times 0.3000) = -0.1152$$

$$V[1] = (-0.5758 \times 0.1000) + (1.7275 \times 0.2000) + (-0.5758 \times 0.3000) + (-0.5758 \times 0.0000) = 0.1152$$

I (Xn row = $[1.6458, -0.1097, -0.9875, -0.5486]$)

$$V[0] = (1.6458 \times 0.2000) + (-0.1097 \times 0.1000) + (-0.9875 \times 0.0000) + (-0.5486 \times 0.3000) = 0.1536$$

$$V[1] = (1.6458 \times 0.1000) + (-0.1097 \times 0.2000) + (-0.9875 \times 0.3000) + (-0.5486 \times 0.0000) = -0.1536$$

love (Xn row = $[-1.4737, 1.3333, -0.0702, 0.2105]$)

$$V[0] = (-1.4737 \times 0.2000) + (1.3333 \times 0.1000) + (-0.0702 \times 0.0000) + (0.2105 \times 0.3000) = -0.0982$$

$$V[1] = (-1.4737 \times 0.1000) + (1.3333 \times 0.2000) + (-0.0702 \times 0.3000) + (0.2105 \times 0.0000) = 0.0982$$

AI (Xn row = $[0.9999, -1.6665, 0.3333, 0.3333]$)

$$V[0] = (0.9999 \times 0.2000) + (-1.6665 \times 0.1000) + (0.3333 \times 0.0000) + (0.3333 \times 0.3000) = 0.1333$$

$$V[1] = (0.9999 \times 0.1000) + (-1.6665 \times 0.2000) + (0.3333 \times 0.3000) + (0.3333 \times 0.0000) = -0.1333$$

Resulting V matrix (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & -0.1152 & 0.1152 \\ \text{I} & 0.1536 & -0.1536 \\ \text{love} & -0.0982 & 0.0982 \\ \text{AI} & 0.1333 & -0.1333 \end{array}$$

Step 6 (Head 2): standard Q , K , V projections (direct, NO compression)

W_{q_2} (shape 4×2):

$$\begin{bmatrix} 0.0000 & 1.0000 \\ 1.0000 & 0.0000 \\ 0.0000 & 0.5000 \\ 0.5000 & 0.0000 \end{bmatrix}$$

X_n (**every token as a row**) (shape 4×4):

$$\begin{array}{c|cccc} \langle \text{bos} \rangle & -0.5758 & 1.7275 & -0.5758 & -0.5758 \\ \text{I} & 1.6458 & -0.1097 & -0.9875 & -0.5486 \\ \text{love} & -1.4737 & 1.3333 & -0.0702 & 0.2105 \\ \text{AI} & 0.9999 & -1.6665 & 0.3333 & 0.3333 \end{array}$$

Expanded arithmetic, $Q = X_n \cdot W_{q_2}$, every token:

$\langle \text{bos} \rangle$ (X_n row = $[-0.5758, 1.7275, -0.5758, -0.5758]$)

$$Q[0] = (-0.5758 \times 0.0000) + (1.7275 \times 1.0000) + (-0.5758 \times 0.0000) + (-0.5758 \times 0.5000) = 1.4395$$

$$Q[1] = (-0.5758 \times 1.0000) + (1.7275 \times 0.0000) + (-0.5758 \times 0.5000) + (-0.5758 \times 0.0000) = -0.8637$$

I (X_n row = $[1.6458, -0.1097, -0.9875, -0.5486]$)

$$Q[0] = (1.6458 \times 0.0000) + (-0.1097 \times 1.0000) + (-0.9875 \times 0.0000) + (-0.5486 \times 0.5000) = -0.3840$$

$$Q[1] = (1.6458 \times 1.0000) + (-0.1097 \times 0.0000) + (-0.9875 \times 0.5000) + (-0.5486 \times 0.0000) = 1.1521$$

love (X_n row = $[-1.4737, 1.3333, -0.0702, 0.2105]$)

$$Q[0] = (-1.4737 \times 0.0000) + (1.3333 \times 1.0000) + (-0.0702 \times 0.0000) + (0.2105 \times 0.5000) = 1.4386$$

$$Q[1] = (-1.4737 \times 1.0000) + (1.3333 \times 0.0000) + (-0.0702 \times 0.5000) + (0.2105 \times 0.0000) = -1.5088$$

AI (X_n row = $[0.9999, -1.6665, 0.3333, 0.3333]$)

$$Q[0] = (0.9999 \times 0.0000) + (-1.6665 \times 1.0000) + (0.3333 \times 0.0000) + (0.3333 \times 0.5000) = -1.4999$$

$$Q[1] = (0.9999 \times 1.0000) + (-1.6665 \times 0.0000) + (0.3333 \times 0.5000) + (0.3333 \times 0.0000) = 1.1666$$

Resulting Q matrix (shape 4×2):

<bos>	1.4395	-0.8637
I	-0.3840	1.1521
love	1.4386	-1.5088
AI	-1.4999	1.1666

Wk_2 (shape 4×2):

0.0000	0.8000
0.8000	0.0000
0.0000	0.3000
0.3000	0.0000

Xn (**every token as a row**) (shape 4×4):

<bos>	-0.5758	1.7275	-0.5758	-0.5758
I	1.6458	-0.1097	-0.9875	-0.5486
love	-1.4737	1.3333	-0.0702	0.2105
AI	0.9999	-1.6665	0.3333	0.3333

Expanded arithmetic, $K = Xn \cdot Wk_2$, every token:

<bos> (Xn row = $[-0.5758, 1.7275, -0.5758, -0.5758]$)

$$K[0] = (-0.5758 \times 0.0000) + (1.7275 \times 0.8000) + (-0.5758 \times 0.0000) + (-0.5758 \times 0.3000) = 1.2092$$

$$K[1] = (-0.5758 \times 0.8000) + (1.7275 \times 0.0000) + (-0.5758 \times 0.3000) + (-0.5758 \times 0.0000) = -0.6334$$

I (Xn row = $[1.6458, -0.1097, -0.9875, -0.5486]$)

$$K[0] = (1.6458 \times 0.0000) + (-0.1097 \times 0.8000) + (-0.9875 \times 0.0000) + (-0.5486 \times 0.3000) = -0.2524$$

$$K[1] = (1.6458 \times 0.8000) + (-0.1097 \times 0.0000) + (-0.9875 \times 0.3000) + (-0.5486 \times 0.0000) = 1.0204$$

love (Xn row = $[-1.4737, 1.3333, -0.0702, 0.2105]$)

$$K[0] = (-1.4737 \times 0.0000) + (1.3333 \times 0.8000) + (-0.0702 \times 0.0000) + (0.2105 \times 0.3000) = 1.1298$$

$$K[1] = (-1.4737 \times 0.8000) + (1.3333 \times 0.0000) + (-0.0702 \times 0.3000) + (0.2105 \times 0.0000) = -1.2000$$

AI (Xn row = $[0.9999, -1.6665, 0.3333, 0.3333]$)

$$K[0] = (0.9999 \times 0.0000) + (-1.6665 \times 0.8000) + (0.3333 \times 0.0000) + (0.3333 \times 0.3000) = -1.2332$$

$$K[1] = (0.9999 \times 0.8000) + (-1.6665 \times 0.0000) + (0.3333 \times 0.3000) + (0.3333 \times 0.0000) = 0.8999$$

Resulting K matrix (shape 4×2):

<bos>	1.2092	-0.6334
I	-0.2524	1.0204
love	1.1298	-1.2000
AI	-1.2332	0.8999

Wv_2 (shape 4×2):

$$\begin{bmatrix} 0.1000 & 0.2000 \\ 0.2000 & 0.1000 \\ 0.3000 & 0.0000 \\ 0.0000 & 0.3000 \end{bmatrix}$$

Xn (**every token as a row**) (shape 4×4):

$$\begin{array}{c|cccc} \langle \text{bos} \rangle & -0.5758 & 1.7275 & -0.5758 & -0.5758 \\ \text{I} & 1.6458 & -0.1097 & -0.9875 & -0.5486 \\ \text{love} & -1.4737 & 1.3333 & -0.0702 & 0.2105 \\ \text{AI} & 0.9999 & -1.6665 & 0.3333 & 0.3333 \end{array}$$

Expanded arithmetic, $V = Xn \cdot Wv_2$, every token:

$\langle \text{bos} \rangle$ (Xn row = $[-0.5758, 1.7275, -0.5758, -0.5758]$)

$$V[0] = (-0.5758 \times 0.1000) + (1.7275 \times 0.2000) + (-0.5758 \times 0.3000) + (-0.5758 \times 0.0000) = 0.1152$$

$$V[1] = (-0.5758 \times 0.2000) + (1.7275 \times 0.1000) + (-0.5758 \times 0.0000) + (-0.5758 \times 0.3000) = -0.1152$$

I (Xn row = $[1.6458, -0.1097, -0.9875, -0.5486]$)

$$V[0] = (1.6458 \times 0.1000) + (-0.1097 \times 0.2000) + (-0.9875 \times 0.3000) + (-0.5486 \times 0.0000) = -0.1536$$

$$V[1] = (1.6458 \times 0.2000) + (-0.1097 \times 0.1000) + (-0.9875 \times 0.0000) + (-0.5486 \times 0.3000) = 0.1536$$

love (Xn row = $[-1.4737, 1.3333, -0.0702, 0.2105]$)

$$V[0] = (-1.4737 \times 0.1000) + (1.3333 \times 0.2000) + (-0.0702 \times 0.3000) + (0.2105 \times 0.0000) = 0.0982$$

$$V[1] = (-1.4737 \times 0.2000) + (1.3333 \times 0.1000) + (-0.0702 \times 0.0000) + (0.2105 \times 0.3000) = -0.0982$$

AI (Xn row = $[0.9999, -1.6665, 0.3333, 0.3333]$)

$$V[0] = (0.9999 \times 0.1000) + (-1.6665 \times 0.2000) + (0.3333 \times 0.3000) + (0.3333 \times 0.0000) = -0.1333$$

$$V[1] = (0.9999 \times 0.2000) + (-1.6665 \times 0.1000) + (0.3333 \times 0.0000) + (0.3333 \times 0.3000) = 0.1333$$

Resulting V matrix (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.1152 & -0.1152 \\ \text{I} & -0.1536 & 0.1536 \\ \text{love} & 0.0982 & -0.0982 \\ \text{AI} & -0.1333 & 0.1333 \end{array}$$

Step 7 (Head 1): full causal scaled dot-product attention

Head 1: Q matrix (shape 4×2):

<bos>	−0.8637	1.4395
I	1.1521	−0.3840
love	−1.5088	1.4386
AI	1.1666	−1.4999

Head 1: K matrix (shape 4×2):

<bos>	−0.6334	1.2092
I	1.0204	−0.2524
love	−1.2000	1.1298
AI	0.8999	−1.2332

Head 1: V matrix (shape 4×2):

<bos>	−0.1152	0.1152
I	0.1536	−0.1536
love	−0.0982	0.0982
AI	0.1333	−0.1333

Causal attention scores $\text{score}(i, j) = (Q_i \cdot K_j) / \sqrt{2} = (Q_i \cdot K_j) / 1.4142$, only for $j \leq i$:

Query: <bos> (position 0, sees positions 0..0)

$$\text{score}(\text{<bos>, <bos>}) = \frac{(-0.8637 \times -0.6334) + (1.4395 \times 1.2092)}{1.4142} = \frac{2.2878}{1.4142} = 1.6177$$

Query: I (position 1, sees positions 0..1)

$$\begin{aligned} \text{score}(\text{I, <bos>}) &= \frac{(1.1521 \times -0.6334) + (-0.3840 \times 1.2092)}{1.4142} = \frac{-1.1941}{1.4142} = -0.8444 \\ \text{score}(\text{I, I}) &= \frac{(1.1521 \times 1.0204) + (-0.3840 \times -0.2524)}{1.4142} = \frac{1.2725}{1.4142} = 0.8998 \end{aligned}$$

Query: love (position 2, sees positions 0..2)

$$\begin{aligned} \text{score}(\text{love, <bos>}) &= \frac{(-1.5088 \times -0.6334) + (1.4386 \times 1.2092)}{1.4142} = \frac{2.6952}{1.4142} = 1.9058 \\ \text{score}(\text{love, I}) &= \frac{(-1.5088 \times 1.0204) + (1.4386 \times -0.2524)}{1.4142} = \frac{-1.9026}{1.4142} = -1.3453 \\ \text{score}(\text{love, love}) &= \frac{(-1.5088 \times -1.2000) + (1.4386 \times 1.1298)}{1.4142} = \frac{3.4359}{1.4142} = 2.4295 \end{aligned}$$

Query: AI (position 3, sees positions 0..3)

$$\begin{aligned} \text{score}(\text{AI, <bos>}) &= \frac{(1.1666 \times -0.6334) + (-1.4999 \times 1.2092)}{1.4142} = \frac{-2.5525}{1.4142} = -1.8049 \\ \text{score}(\text{AI, I}) &= \frac{(1.1666 \times 1.0204) + (-1.4999 \times -0.2524)}{1.4142} = \frac{1.5689}{1.4142} = 1.1094 \end{aligned}$$

$$\begin{aligned}\text{score}(\text{AI}, \text{love}) &= \frac{(1.1666 \times -1.2000) + (-1.4999 \times 1.1298)}{1.4142} = \frac{-3.0944}{1.4142} = -2.1881 \\ \text{score}(\text{AI}, \text{AI}) &= \frac{(1.1666 \times 0.8999) + (-1.4999 \times -1.2332)}{1.4142} = \frac{2.8994}{1.4142} = 2.0502\end{aligned}$$

Softmax over valid (unmasked) positions, every row:

$$\langle \text{bos} \rangle: \text{scores} = [1.6177]$$

$$\text{exponentials} = [e^{1.6177} = 1.0000]$$

$$\text{sum} = 1.0000$$

$$\text{weights} = [\frac{1.0000}{1.0000} = 1.0000]$$

$$I: \text{scores} = [-0.8444, 0.8998]$$

$$\text{exponentials} = [e^{-0.8444} = 0.1748, e^{0.8998} = 1.0000]$$

$$\text{sum} = 1.1748$$

$$\text{weights} = [\frac{0.1748}{1.1748} = 0.1488, \frac{1.0000}{1.1748} = 0.8512]$$

$$\text{love}: \text{scores} = [1.9058, -1.3453, 2.4295]$$

$$\text{exponentials} = [e^{1.9058} = 0.5923, e^{-1.3453} = 0.0229, e^{2.4295} = 1.0000]$$

$$\text{sum} = 1.6153$$

$$\text{weights} = [\frac{0.5923}{1.6153} = 0.3667, \frac{0.0229}{1.6153} = 0.0142, \frac{1.0000}{1.6153} = 0.6191]$$

$$AI: \text{scores} = [-1.8049, 1.1094, -2.1881, 2.0502]$$

$$\text{exponentials} = [e^{-1.8049} = 0.0212, e^{1.1094} = 0.3903, e^{-2.1881} = 0.0144, e^{2.0502} = 1.0000]$$

$$\text{sum} = 1.4259$$

$$\text{weights} = [\frac{0.0212}{1.4259} = 0.0148, \frac{0.3903}{1.4259} = 0.2737, \frac{0.0144}{1.4259} = 0.0101, \frac{1.0000}{1.4259} = 0.7013]$$

Attention output = weighted sum of V, every token:

$$\langle \text{bos} \rangle_{[0]} = (1.0000 \times -0.1152) = -0.1152$$

$$\langle \text{bos} \rangle_{[1]} = (1.0000 \times 0.1152) = 0.1152$$

$$I_{[0]} = (0.1488 \times -0.1152) + (0.8512 \times 0.1536) = 0.1136$$

$$I_{[1]} = (0.1488 \times 0.1152) + (0.8512 \times -0.1536) = -0.1136$$

$$\text{love}_{[0]} = (0.3667 \times -0.1152) + (0.0142 \times 0.1536) + (0.6191 \times -0.0982) = -0.1009$$

$$\text{love}_{[1]} = (0.3667 \times 0.1152) + (0.0142 \times -0.1536) + (0.6191 \times 0.0982) = 0.1009$$

$$AI_{[0]} = (0.0148 \times -0.1152) + (0.2737 \times 0.1536) + (0.0101 \times -0.0982) + (0.7013 \times 0.1333) = 0.1328$$

$$AI_{[1]} = (0.0148 \times 0.1152) + (0.2737 \times -0.1536) + (0.0101 \times 0.0982) + (0.7013 \times -0.1333) = -0.1328$$

Head 1: resulting attention-output matrix (shape 4×2):

$$\begin{array}{c|cc} <\text{bos}> & -0.1152 & 0.1152 \\ \text{I} & 0.1136 & -0.1136 \\ \text{love} & -0.1009 & 0.1009 \\ \text{AI} & 0.1328 & -0.1328 \end{array}$$

Step 7 (Head 2): full causal scaled dot-product attention

Head 2: Q matrix (shape 4×2):

$$\begin{array}{c|cc} <\text{bos}> & 1.4395 & -0.8637 \\ \text{I} & -0.3840 & 1.1521 \\ \text{love} & 1.4386 & -1.5088 \\ \text{AI} & -1.4999 & 1.1666 \end{array}$$

Head 2: K matrix (shape 4×2):

$$\begin{array}{c|cc} <\text{bos}> & 1.2092 & -0.6334 \\ \text{I} & -0.2524 & 1.0204 \\ \text{love} & 1.1298 & -1.2000 \\ \text{AI} & -1.2332 & 0.8999 \end{array}$$

Head 2: V matrix (shape 4×2):

$$\begin{array}{c|cc} <\text{bos}> & 0.1152 & -0.1152 \\ \text{I} & -0.1536 & 0.1536 \\ \text{love} & 0.0982 & -0.0982 \\ \text{AI} & -0.1333 & 0.1333 \end{array}$$

Causal attention scores $\text{score}(i, j) = (Q_i \cdot K_j) / \sqrt{2} = (Q_i \cdot K_j) / 1.4142$, only for $j \leq i$:

Query: **<bos>** (position 0, sees positions 0..0)

$$\text{score}(<\text{bos}>, <\text{bos}>) = \frac{(1.4395 \times 1.2092) + (-0.8637 \times -0.6334)}{1.4142} = \frac{2.2878}{1.4142} = 1.6177$$

Query: **I** (position 1, sees positions 0..1)

$$\begin{aligned} \text{score}(\text{I}, <\text{bos}>) &= \frac{(-0.3840 \times 1.2092) + (1.1521 \times -0.6334)}{1.4142} = \frac{-1.1941}{1.4142} = -0.8444 \\ \text{score}(\text{I}, \text{I}) &= \frac{(-0.3840 \times -0.2524) + (1.1521 \times 1.0204)}{1.4142} = \frac{1.2725}{1.4142} = 0.8998 \end{aligned}$$

Query: **love** (position 2, sees positions 0..2)

$$\text{score}(\text{love}, <\text{bos}>) = \frac{(1.4386 \times 1.2092) + (-1.5088 \times -0.6334)}{1.4142} = \frac{2.6952}{1.4142} = 1.9058$$

$$\begin{aligned}\text{score}(\text{love}, \text{I}) &= \frac{(1.4386 \times -0.2524) + (-1.5088 \times 1.0204)}{1.4142} = \frac{-1.9026}{1.4142} = -1.3453 \\ \text{score}(\text{love}, \text{love}) &= \frac{(1.4386 \times 1.1298) + (-1.5088 \times -1.2000)}{1.4142} = \frac{3.4359}{1.4142} = 2.4295\end{aligned}$$

Query: AI (position 3, sees positions 0..3)

$$\begin{aligned}\text{score}(\text{AI}, <\text{bos}>) &= \frac{(-1.4999 \times 1.2092) + (1.1666 \times -0.6334)}{1.4142} = \frac{-2.5525}{1.4142} = -1.8049 \\ \text{score}(\text{AI}, \text{I}) &= \frac{(-1.4999 \times -0.2524) + (1.1666 \times 1.0204)}{1.4142} = \frac{1.5689}{1.4142} = 1.1094 \\ \text{score}(\text{AI}, \text{love}) &= \frac{(-1.4999 \times 1.1298) + (1.1666 \times -1.2000)}{1.4142} = \frac{-3.0944}{1.4142} = -2.1881 \\ \text{score}(\text{AI}, \text{AI}) &= \frac{(-1.4999 \times -1.2332) + (1.1666 \times 0.8999)}{1.4142} = \frac{2.8994}{1.4142} = 2.0502\end{aligned}$$

Softmax over valid (unmasked) positions, every row:

<bos>: scores = [1.6177]

$$\begin{aligned}\text{exponentials} &= [e^{1.6177} = 1.0000] \\ \text{sum} &= 1.0000 \\ \text{weights} &= [\frac{1.0000}{1.0000} = 1.0000]\end{aligned}$$

I: scores = [-0.8444, 0.8998]

$$\begin{aligned}\text{exponentials} &= [e^{-0.8444} = 0.1748, e^{0.8998} = 1.0000] \\ \text{sum} &= 1.1748 \\ \text{weights} &= [\frac{0.1748}{1.1748} = 0.1488, \frac{1.0000}{1.1748} = 0.8512]\end{aligned}$$

love: scores = [1.9058, -1.3453, 2.4295]

$$\begin{aligned}\text{exponentials} &= [e^{1.9058} = 0.5923, e^{-1.3453} = 0.0229, e^{2.4295} = 1.0000] \\ \text{sum} &= 1.6153 \\ \text{weights} &= [\frac{0.5923}{1.6153} = 0.3667, \frac{0.0229}{1.6153} = 0.0142, \frac{1.0000}{1.6153} = 0.6191]\end{aligned}$$

AI: scores = [-1.8049, 1.1094, -2.1881, 2.0502]

$$\begin{aligned}\text{exponentials} &= [e^{-1.8049} = 0.0212, e^{1.1094} = 0.3903, e^{-2.1881} = 0.0144, e^{2.0502} = 1.0000] \\ \text{sum} &= 1.4259 \\ \text{weights} &= [\frac{0.0212}{1.4259} = 0.0148, \frac{0.3903}{1.4259} = 0.2737, \frac{0.0144}{1.4259} = 0.0101, \frac{1.0000}{1.4259} = 0.7013]\end{aligned}$$

Attention output = weighted sum of V, every token:

$$<\text{bos}>_{[0]} = (1.0000 \times 0.1152) = 0.1152$$

$$\langle \text{bos} \rangle_{[1]} = (1.0000 \times -0.1152) = -0.1152$$

$$I_{[0]} = (0.1488 \times 0.1152) + (0.8512 \times -0.1536) = -0.1136$$

$$I_{[1]} = (0.1488 \times -0.1152) + (0.8512 \times 0.1536) = 0.1136$$

$$\text{love}_{[0]} = (0.3667 \times 0.1152) + (0.0142 \times -0.1536) + (0.6191 \times 0.0982) = 0.1009$$

$$\text{love}_{[1]} = (0.3667 \times -0.1152) + (0.0142 \times 0.1536) + (0.6191 \times -0.0982) = -0.1009$$

$$AI_{[0]} = (0.0148 \times 0.1152) + (0.2737 \times -0.1536) + (0.0101 \times 0.0982) + (0.7013 \times -0.1333) = -0.1328$$

$$AI_{[1]} = (0.0148 \times -0.1152) + (0.2737 \times 0.1536) + (0.0101 \times -0.0982) + (0.7013 \times 0.1333) = 0.1328$$

Head 2: resulting attention-output matrix (shape 4×2):

$\langle \text{bos} \rangle$	0.1152	-0.1152
I	-0.1136	0.1136
love	0.1009	-0.1009
AI	-0.1328	0.1328

Step 8: Concatenate heads, project through W_o

Concat(head1, head2) (shape 4×4):

$\langle \text{bos} \rangle$	-0.1152	0.1152	0.1152	-0.1152
I	0.1136	-0.1136	-0.1136	0.1136
love	-0.1009	0.1009	0.1009	-0.1009
AI	0.1328	-0.1328	-0.1328	0.1328

W_o (shape 4×4):

$$\begin{bmatrix} 0.4000 & 0.1000 & 0.2000 & 0.3000 \\ 0.2000 & 0.3000 & 0.1000 & 0.0000 \\ 0.1000 & 0.2000 & 0.4000 & 0.1000 \\ 0.3000 & 0.0000 & 0.1000 & 0.4000 \end{bmatrix}$$

concat (**every token as a row**) (shape 4×4):

$\langle \text{bos} \rangle$	-0.1152	0.1152	0.1152	-0.1152
I	0.1136	-0.1136	-0.1136	0.1136
love	-0.1009	0.1009	0.1009	-0.1009
AI	0.1328	-0.1328	-0.1328	0.1328

Expanded arithmetic, $MultiHead = concat \cdot W_o$, every token:

$\langle \text{bos} \rangle$ (*concat* row = $[-0.1152, 0.1152, 0.1152, -0.1152]$)

$$MultiHead[0] = (-0.1152 \times 0.4000) + (0.1152 \times 0.2000) + (0.1152 \times 0.1000) + (-0.1152 \times 0.3000) = -0.0461$$

$$\begin{aligned}
MultiHead[1] &= (-0.1152 \times 0.1000) + (0.1152 \times 0.3000) + (0.1152 \times 0.2000) + (-0.1152 \times 0.0000) = 0.0461 \\
MultiHead[2] &= (-0.1152 \times 0.2000) + (0.1152 \times 0.1000) + (0.1152 \times 0.4000) + (-0.1152 \times 0.1000) = 0.0230 \\
MultiHead[3] &= (-0.1152 \times 0.3000) + (0.1152 \times 0.0000) + (0.1152 \times 0.1000) + (-0.1152 \times 0.4000) = -0.0691 \\
I \text{ (concat row} &= [0.1136, -0.1136, -0.1136, 0.1136])
\end{aligned}$$

$$\begin{aligned}
MultiHead[0] &= (0.1136 \times 0.4000) + (-0.1136 \times 0.2000) + (-0.1136 \times 0.1000) + (0.1136 \times 0.3000) = 0.0454 \\
MultiHead[1] &= (0.1136 \times 0.1000) + (-0.1136 \times 0.3000) + (-0.1136 \times 0.2000) + (0.1136 \times 0.0000) = -0.0454 \\
MultiHead[2] &= (0.1136 \times 0.2000) + (-0.1136 \times 0.1000) + (-0.1136 \times 0.4000) + (0.1136 \times 0.1000) = -0.0227 \\
MultiHead[3] &= (0.1136 \times 0.3000) + (-0.1136 \times 0.0000) + (-0.1136 \times 0.1000) + (0.1136 \times 0.4000) = 0.0682 \\
love \text{ (concat row} &= [-0.1009, 0.1009, 0.1009, -0.1009])
\end{aligned}$$

$$\begin{aligned}
MultiHead[0] &= (-0.1009 \times 0.4000) + (0.1009 \times 0.2000) + (0.1009 \times 0.1000) + (-0.1009 \times 0.3000) = -0.0403 \\
MultiHead[1] &= (-0.1009 \times 0.1000) + (0.1009 \times 0.3000) + (0.1009 \times 0.2000) + (-0.1009 \times 0.0000) = 0.0403 \\
MultiHead[2] &= (-0.1009 \times 0.2000) + (0.1009 \times 0.1000) + (0.1009 \times 0.4000) + (-0.1009 \times 0.1000) = 0.0202 \\
MultiHead[3] &= (-0.1009 \times 0.3000) + (0.1009 \times 0.0000) + (0.1009 \times 0.1000) + (-0.1009 \times 0.4000) = -0.0605 \\
AI \text{ (concat row} &= [0.1328, -0.1328, -0.1328, 0.1328])
\end{aligned}$$

$$\begin{aligned}
MultiHead[0] &= (0.1328 \times 0.4000) + (-0.1328 \times 0.2000) + (-0.1328 \times 0.1000) + (0.1328 \times 0.3000) = 0.0531 \\
MultiHead[1] &= (0.1328 \times 0.1000) + (-0.1328 \times 0.3000) + (-0.1328 \times 0.2000) + (0.1328 \times 0.0000) = -0.0531 \\
MultiHead[2] &= (0.1328 \times 0.2000) + (-0.1328 \times 0.1000) + (-0.1328 \times 0.4000) + (0.1328 \times 0.1000) = -0.0266 \\
MultiHead[3] &= (0.1328 \times 0.3000) + (-0.1328 \times 0.0000) + (-0.1328 \times 0.1000) + (0.1328 \times 0.4000) = 0.0797
\end{aligned}$$

Resulting *MultiHead* matrix (shape 4×4):

$$\begin{array}{c|cccc}
<bos> & -0.0461 & 0.0461 & 0.0230 & -0.0691 \\
I & 0.0454 & -0.0454 & -0.0227 & 0.0682 \\
love & -0.0403 & 0.0403 & 0.0202 & -0.0605 \\
AI & 0.0531 & -0.0531 & -0.0266 & 0.0797
\end{array}$$

Step 9: Residual add $Y = X + \text{MultiHead}(X)$

X (original, pre-normalization) (shape 4×4):

$$\begin{array}{c|cccc}
<bos> & 0.0500 & 0.1500 & 0.0500 & 0.0500 \\
I & 0.3000 & 0.1000 & 0.0000 & 0.0500 \\
love & -0.1000 & 0.4000 & 0.1500 & 0.2000 \\
AI & 0.3500 & -0.2500 & 0.2000 & 0.2000
\end{array}$$

MultiHead(X) (shape 4×4):

<bos>	-0.0461	0.0461	0.0230	-0.0691
I	0.0454	-0.0454	-0.0227	0.0682
love	-0.0403	0.0403	0.0202	-0.0605
AI	0.0531	-0.0531	-0.0266	0.0797

Resulting Y matrix (shape 4×4):

<bos>	0.0039	0.1961	0.0730	-0.0191
I	0.3454	0.0546	-0.0227	0.1182
love	-0.1403	0.4403	0.1702	0.1395
AI	0.4031	-0.3031	0.1734	0.2797

Step 10: Second LayerNorm (pre-FFN)

Input matrix (every token as a row) (shape 4×4):

<bos>	0.0039	0.1961	0.0730	-0.0191
I	0.3454	0.0546	-0.0227	0.1182
love	-0.1403	0.4403	0.1702	0.1395
AI	0.4031	-0.3031	0.1734	0.2797

$\gamma = 1, \beta = 0$ (identity affine, to isolate the normalization math)

<bos>: $v = [0.0039, 0.1961, 0.0730, -0.0191]$

$$\text{mean} = \frac{0.0039 + 0.1961 + 0.0730 + -0.0191}{4} = 0.0635$$

$$\text{deviations} = [-0.0595, 0.1326, 0.0095, -0.0826] \Rightarrow \text{variance} = \frac{0.0280}{4} = 0.0070$$

$$\text{std} = \sqrt{0.0070 + 10^{-5}} = 0.0838$$

$$\text{normalized} = \left[\frac{-0.0595}{0.0838} = -0.7108, \frac{0.1326}{0.0838} = 1.5825, \frac{0.0095}{0.0838} = 0.1140, \frac{-0.0826}{0.0838} = -0.9857 \right]$$

I: $v = [0.3454, 0.0546, -0.0227, 0.1182]$

$$\text{mean} = \frac{0.3454 + 0.0546 + -0.0227 + 0.1182}{4} = 0.1239$$

$$\text{deviations} = [0.2216, -0.0693, -0.1466, -0.0057] \Rightarrow \text{variance} = \frac{0.0754}{4} = 0.0189$$

$$\text{std} = \sqrt{0.0189 + 10^{-5}} = 0.1374$$

$$\text{normalized} = \left[\frac{0.2216}{0.1374} = 1.6133, \frac{-0.0693}{0.1374} = -0.5046, \frac{-0.1466}{0.1374} = -1.0672, \frac{-0.0057}{0.1374} = -0.0414 \right]$$

love: $v = [-0.1403, 0.4403, 0.1702, 0.1395]$

$$\text{mean} = \frac{-0.1403 + 0.4403 + 0.1702 + 0.1395}{4} = 0.1524$$

$$\text{deviations} = [-0.2928, 0.2879, 0.0178, -0.0129] \Rightarrow \text{variance} = \frac{0.1691}{4} = 0.0423$$

$$\text{std} = \sqrt{0.0423 + 10^{-5}} = 0.2056$$

$$\text{normalized} = [\frac{-0.2928}{0.2056} = -1.4237, \frac{0.2879}{0.2056} = 1.4002, \frac{0.0178}{0.2056} = 0.0864, \frac{-0.0129}{0.2056} = -0.0629]$$

$$AI: v = [0.4031, -0.3031, 0.1734, 0.2797]$$

$$\text{mean} = \frac{0.4031 + -0.3031 + 0.1734 + 0.2797}{4} = 0.1383$$

$$\text{deviations} = [0.2649, -0.4414, 0.0351, 0.1414] \Rightarrow \text{variance} = \frac{0.2862}{4} = 0.0716$$

$$\text{std} = \sqrt{0.0716 + 10^{-5}} = 0.2675$$

$$\text{normalized} = [\frac{0.2649}{0.2675} = 0.9900, \frac{-0.4414}{0.2675} = -1.6500, \frac{0.0351}{0.2675} = 0.1314, \frac{0.1414}{0.2675} = 0.5286]$$

Resulting normalized matrix (shape 4×4):

$\langle \text{bos} \rangle$	-0.7108	1.5825	0.1140	-0.9857
I	1.6133	-0.5046	-1.0672	-0.0414
love	-1.4237	1.4002	0.0864	-0.0629
AI	0.9900	-1.6500	0.1314	0.5286

Step 11: Standard dense FFN — first linear layer (no experts, no routing)

W_1 (shape 4×8):

0.0300	0.1250	-0.1250	0.0060	0.2360	0.2380	-0.2250	-0.1760
-0.2690	-0.0360	-0.2820	-0.0260	0.0890	-0.1330	0.1060	0.0550
-0.2860	0.0350	-0.1440	-0.0510	-0.1300	0.1160	-0.0360	-0.2060
0.0270	0.1680	-0.1160	-0.1670	-0.0670	0.2620	0.2860	0.1030

Yn (**every token as a row**) (shape 4×4):

$\langle \text{bos} \rangle$	-0.7108	1.5825	0.1140	-0.9857
I	1.6133	-0.5046	-1.0672	-0.0414
love	-1.4237	1.4002	0.0864	-0.0629
AI	0.9900	-1.6500	0.1314	0.5286

Expanded arithmetic, $hidden = Yn \cdot W_1$, every token:

$$\langle \text{bos} \rangle (Yn \text{ row} = [-0.7108, 1.5825, 0.1140, -0.9857])$$

$$hidden[0] = (-0.7108 \times 0.0300) + (1.5825 \times -0.2690) + (0.1140 \times -0.2860) + (-0.9857 \times 0.0270) = -0.5062$$

$$hidden[1] = (-0.7108 \times 0.1250) + (1.5825 \times -0.0360) + (0.1140 \times 0.0350) + (-0.9857 \times 0.1680) = -0.3074$$

$$hidden[2] = (-0.7108 \times -0.1250) + (1.5825 \times -0.2820) + (0.1140 \times -0.1440) + (-0.9857 \times -0.1160) = -0.2595$$

$$hidden[3] = (-0.7108 \times 0.0060) + (1.5825 \times -0.0260) + (0.1140 \times -0.0510) + (-0.9857 \times -0.1670) = 0.1134$$

$$hidden[4] = (-0.7108 \times 0.2360) + (1.5825 \times 0.0890) + (0.1140 \times -0.1300) + (-0.9857 \times -0.0670) = 0.0243$$

$$hidden[5] = (-0.7108 \times 0.2380) + (1.5825 \times -0.1330) + (0.1140 \times 0.1160) + (-0.9857 \times 0.2620) = -0.6247$$

$$hidden[6] = (-0.7108 \times -0.2250) + (1.5825 \times 0.1060) + (0.1140 \times -0.0360) + (-0.9857 \times 0.2860) = 0.0417$$

$$hidden[7] = (-0.7108 \times -0.1760) + (1.5825 \times 0.0550) + (0.1140 \times -0.2060) + (-0.9857 \times 0.1030) = 0.0871$$

I (Yn row = [1.6133, -0.5046, -1.0672, -0.0414])

$$hidden[0] = (1.6133 \times 0.0300) + (-0.5046 \times -0.2690) + (-1.0672 \times -0.2860) + (-0.0414 \times 0.0270) = 0.4882$$

$$hidden[1] = (1.6133 \times 0.1250) + (-0.5046 \times -0.0360) + (-1.0672 \times 0.0350) + (-0.0414 \times 0.1680) = 0.1755$$

$$hidden[2] = (1.6133 \times -0.1250) + (-0.5046 \times -0.2820) + (-1.0672 \times -0.1440) + (-0.0414 \times -0.1160) = 0.0991$$

$$hidden[3] = (1.6133 \times 0.0060) + (-0.5046 \times -0.0260) + (-1.0672 \times -0.0510) + (-0.0414 \times -0.1670) = 0.0841$$

$$hidden[4] = (1.6133 \times 0.2360) + (-0.5046 \times 0.0890) + (-1.0672 \times -0.1300) + (-0.0414 \times -0.0670) = 0.4773$$

$$hidden[5] = (1.6133 \times 0.2380) + (-0.5046 \times -0.1330) + (-1.0672 \times 0.1160) + (-0.0414 \times 0.2620) = 0.3164$$

$$hidden[6] = (1.6133 \times -0.2250) + (-0.5046 \times 0.1060) + (-1.0672 \times -0.0360) + (-0.0414 \times 0.2860) = -0.3899$$

$$hidden[7] = (1.6133 \times -0.1760) + (-0.5046 \times 0.0550) + (-1.0672 \times -0.2060) + (-0.0414 \times 0.1030) = -0.0961$$

$love$ (Yn row = [-1.4237, 1.4002, 0.0864, -0.0629])

$$hidden[0] = (-1.4237 \times 0.0300) + (1.4002 \times -0.2690) + (0.0864 \times -0.2860) + (-0.0629 \times 0.0270) = -0.4458$$

$$hidden[1] = (-1.4237 \times 0.1250) + (1.4002 \times -0.0360) + (0.0864 \times 0.0350) + (-0.0629 \times 0.1680) = -0.2359$$

$$hidden[2] = (-1.4237 \times -0.1250) + (1.4002 \times -0.2820) + (0.0864 \times -0.1440) + (-0.0629 \times -0.1160) = -0.2220$$

$$hidden[3] = (-1.4237 \times 0.0060) + (1.4002 \times -0.0260) + (0.0864 \times -0.0510) + (-0.0629 \times -0.1670) = -0.0388$$

$$hidden[4] = (-1.4237 \times 0.2360) + (1.4002 \times 0.0890) + (0.0864 \times -0.1300) + (-0.0629 \times -0.0670) = -0.2184$$

$$hidden[5] = (-1.4237 \times 0.2380) + (1.4002 \times -0.1330) + (0.0864 \times 0.1160) + (-0.0629 \times 0.2620) = -0.5315$$

$$hidden[6] = (-1.4237 \times -0.2250) + (1.4002 \times 0.1060) + (0.0864 \times -0.0360) + (-0.0629 \times 0.2860) = 0.4477$$

$$hidden[7] = (-1.4237 \times -0.1760) + (1.4002 \times 0.0550) + (0.0864 \times -0.2060) + (-0.0629 \times 0.1030) = 0.3033$$

AI (Yn row = [0.9900, -1.6500, 0.1314, 0.5286])

$$hidden[0] = (0.9900 \times 0.0300) + (-1.6500 \times -0.2690) + (0.1314 \times -0.2860) + (0.5286 \times 0.0270) = 0.4503$$

$$hidden[1] = (0.9900 \times 0.1250) + (-1.6500 \times -0.0360) + (0.1314 \times 0.0350) + (0.5286 \times 0.1680) = 0.2766$$

$$hidden[2] = (0.9900 \times -0.1250) + (-1.6500 \times -0.2820) + (0.1314 \times -0.1440) + (0.5286 \times -0.1160) = 0.2613$$

$$hidden[3] = (0.9900 \times 0.0060) + (-1.6500 \times -0.0260) + (0.1314 \times -0.0510) + (0.5286 \times -0.1670) = -0.0461$$

$$hidden[4] = (0.9900 \times 0.2360) + (-1.6500 \times 0.0890) + (0.1314 \times -0.1300) + (0.5286 \times -0.0670) = 0.0343$$

$$hidden[5] = (0.9900 \times 0.2380) + (-1.6500 \times -0.1330) + (0.1314 \times 0.1160) + (0.5286 \times 0.2620) = 0.6088$$

$$hidden[6] = (0.9900 \times -0.2250) + (-1.6500 \times 0.1060) + (0.1314 \times -0.0360) + (0.5286 \times 0.2860) = -0.2512$$

$$hidden[7] = (0.9900 \times -0.1760) + (-1.6500 \times 0.0550) + (0.1314 \times -0.2060) + (0.5286 \times 0.1030) = -0.2376$$

Resulting *hidden* matrix (shape 4 × 8):

<bos>	-0.5062	-0.3074	-0.2595	0.1134	0.0243	-0.6247	0.0417	0.0871
I	0.4882	0.1755	0.0991	0.0841	0.4773	0.3164	-0.3899	-0.0961
love	-0.4458	-0.2359	-0.2220	-0.0388	-0.2184	-0.5315	0.4477	0.3033
AI	0.4503	0.2766	0.2613	-0.0461	0.0343	0.6088	-0.2512	-0.2376

Step 12: GELU activation

Hidden matrix BEFORE GELU (shape 4×8):

<bos>	−0.5062	−0.3074	−0.2595	0.1134	0.0243	−0.6247	0.0417	0.0871
I	0.4882	0.1755	0.0991	0.0841	0.4773	0.3164	−0.3899	−0.0961
love	−0.4458	−0.2359	−0.2220	−0.0388	−0.2184	−0.5315	0.4477	0.3033
AI	0.4503	0.2766	0.2613	−0.0461	0.0343	0.6088	−0.2512	−0.2376

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

<bos>:

[−0.5062 → −0.1551, −0.3074 → −0.1166, −0.2595 → −0.1032, 0.1134 → 0.0618,
0.0243 → 0.0124, −0.6247 → −0.1663, 0.0417 → 0.0215, 0.0871 → 0.0466]

I:

[0.4882 → 0.3356, 0.1755 → 0.1000, 0.0991 → 0.0535, 0.0841 → 0.0449,
0.4773 → 0.3262, 0.3164 → 0.1975, −0.3899 → −0.1358, −0.0961 → −0.0444]

love:

[−0.4458 → −0.1462, −0.2359 → −0.0960, −0.2220 → −0.0915, −0.0388 → −0.0188,
−0.2184 → −0.0903, −0.5315 → −0.1582, 0.4477 → 0.3012, 0.3033 → 0.1878]

AI:

[0.4503 → 0.3033, 0.2766 → 0.1684, 0.2613 → 0.1576, −0.0461 → −0.0222,
0.0343 → 0.0176, 0.6088 → 0.4436, −0.2512 → −0.1007, −0.2376 → −0.0965]

Resulting activated matrix (shape 4×8):

<bos>	−0.1551	−0.1166	−0.1032	0.0618	0.0124	−0.1663	0.0215	0.0466
I	0.3356	0.1000	0.0535	0.0449	0.3262	0.1975	−0.1358	−0.0444
love	−0.1462	−0.0960	−0.0915	−0.0188	−0.0903	−0.1582	0.3012	0.1878
AI	0.3033	0.1684	0.1576	−0.0222	0.0176	0.4436	−0.1007	−0.0965

Step 13: Standard dense FFN — second linear layer

W_2 (shape 8×4):

$$\begin{bmatrix} 0.2800 & 0.0280 & 0.2840 & 0.1290 \\ 0.1190 & −0.1700 & 0.2860 & −0.2960 \\ −0.1480 & −0.0390 & 0.1680 & −0.1810 \\ 0.2180 & 0.2900 & −0.2020 & 0.0580 \\ −0.2950 & −0.0680 & −0.2740 & 0.2740 \\ −0.0380 & 0.2690 & 0.1720 & 0.2200 \\ −0.1960 & −0.2550 & 0.0600 & −0.1990 \\ 0.1400 & −0.0550 & 0.0170 & 0.2630 \end{bmatrix}$$

activated (every token as a row) (shape 4×8):

<bos>	-0.1551	-0.1166	-0.1032	0.0618	0.0124	-0.1663	0.0215	0.0466
I	0.3356	0.1000	0.0535	0.0449	0.3262	0.1975	-0.1358	-0.0444
love	-0.1462	-0.0960	-0.0915	-0.0188	-0.0903	-0.1582	0.3012	0.1878
AI	0.3033	0.1684	0.1576	-0.0222	0.0176	0.4436	-0.1007	-0.0965

Expanded arithmetic, $FFN = activated \cdot W_2$, every token:

<bos> (*activated row* = $[-0.1551, -0.1166, -0.1032, 0.0618, 0.0124, -0.1663, 0.0215, 0.0466]$)

$$FFN[0] = (-0.1551 \times 0.2800) + (-0.1166 \times 0.1190) + (-0.1032 \times -0.1480) + (0.0618 \times 0.2180) \\ + (0.0124 \times -0.2950) + (-0.1663 \times -0.0380) + (0.0215 \times -0.1960) + (0.0466 \times 0.1400) = -0.0236$$

$$FFN[1] = (-0.1551 \times 0.0280) + (-0.1166 \times -0.1700) + (-0.1032 \times -0.0390) + (0.0618 \times 0.2900) \\ + (0.0124 \times -0.0680) + (-0.1663 \times 0.2690) + (0.0215 \times -0.2550) + (0.0466 \times -0.0550) = -0.0162$$

$$FFN[2] = (-0.1551 \times 0.2840) + (-0.1166 \times 0.2860) + (-0.1032 \times 0.1680) + (0.0618 \times -0.2020) \\ + (0.0124 \times -0.2740) + (-0.1663 \times 0.1720) + (0.0215 \times 0.0600) + (0.0466 \times 0.0170) = -0.1371$$

$$FFN[3] = (-0.1551 \times 0.1290) + (-0.1166 \times -0.2960) + (-0.1032 \times -0.1810) + (0.0618 \times 0.0580) \\ + (0.0124 \times 0.2740) + (-0.1663 \times 0.2200) + (0.0215 \times -0.1990) + (0.0466 \times 0.2630) = 0.0116$$

I (*activated row* = $[0.3356, 0.1000, 0.0535, 0.0449, 0.3262, 0.1975, -0.1358, -0.0444]$)

$$FFN[0] = (0.3356 \times 0.2800) + (0.1000 \times 0.1190) + (0.0535 \times -0.1480) + (0.0449 \times 0.2180) \\ + (0.3262 \times -0.2950) + (0.1975 \times -0.0380) + (-0.1358 \times -0.1960) + (-0.0444 \times 0.1400) = 0.0244$$

$$FFN[1] = (0.3356 \times 0.0280) + (0.1000 \times -0.1700) + (0.0535 \times -0.0390) + (0.0449 \times 0.2900) \\ + (0.3262 \times -0.0680) + (0.1975 \times 0.2690) + (-0.1358 \times -0.2550) + (-0.0444 \times -0.0550) = 0.0713$$

$$FFN[2] = (0.3356 \times 0.2840) + (0.1000 \times 0.2860) + (0.0535 \times 0.1680) + (0.0449 \times -0.2020) \\ + (0.3262 \times -0.2740) + (0.1975 \times 0.1720) + (-0.1358 \times 0.0600) + (-0.0444 \times 0.0170) = 0.0595$$

$$FFN[3] = (0.3356 \times 0.1290) + (0.1000 \times -0.2960) + (0.0535 \times -0.1810) + (0.0449 \times 0.0580) \\ + (0.3262 \times 0.2740) + (0.1975 \times 0.2200) + (-0.1358 \times -0.1990) + (-0.0444 \times 0.2630) = 0.1548$$

love (*activated row* = $[-0.1462, -0.0960, -0.0915, -0.0188, -0.0903, -0.1582, 0.3012, 0.1878]$)

$$FFN[0] = (-0.1462 \times 0.2800) + (-0.0960 \times 0.1190) + (-0.0915 \times -0.1480) + (-0.0188 \times 0.2180) \\ + (-0.0903 \times -0.2950) + (-0.1582 \times -0.0380) + (0.3012 \times -0.1960) + (0.1878 \times 0.1400) = -0.0430$$

$$FFN[1] = (-0.1462 \times 0.0280) + (-0.0960 \times -0.1700) + (-0.0915 \times -0.0390) + (-0.0188 \times 0.2900) \\ + (-0.0903 \times -0.0680) + (-0.1582 \times 0.2690) + (0.3012 \times -0.2550) + (0.1878 \times -0.0550) = -0.1132$$

$$FFN[2] = (-0.1462 \times 0.2840) + (-0.0960 \times 0.2860) + (-0.0915 \times 0.1680) + (-0.0188 \times -0.2020) \\ + (-0.0903 \times -0.2740) + (-0.1582 \times 0.1720) + (0.3012 \times 0.0600) + (0.1878 \times 0.0170) = -0.0617$$

$$FFN[3] = (-0.1462 \times 0.1290) + (-0.0960 \times -0.2960) + (-0.0915 \times -0.1810) + (-0.0188 \times 0.0580) \\ + (-0.0903 \times 0.2740) + (-0.1582 \times 0.2200) + (0.3012 \times -0.1990) + (0.1878 \times 0.2630) = -0.0451$$

AI (*activated row* = $[0.3033, 0.1684, 0.1576, -0.0222, 0.0176, 0.4436, -0.1007, -0.0965]$)

$$\begin{aligned}
FFN[0] &= (0.3033 \times 0.2800) + (0.1684 \times 0.1190) + (0.1576 \times -0.1480) + (-0.0222 \times 0.2180) \\
&\quad + (0.0176 \times -0.2950) + (0.4436 \times -0.0380) + (-0.1007 \times -0.1960) + (-0.0965 \times 0.1400) = 0.0610 \\
FFN[1] &= (0.3033 \times 0.0280) + (0.1684 \times -0.1700) + (0.1576 \times -0.0390) + (-0.0222 \times 0.2900) \\
&\quad + (0.0176 \times -0.0680) + (0.4436 \times 0.2690) + (-0.1007 \times -0.2550) + (-0.0965 \times -0.0550) = 0.1164 \\
FFN[2] &= (0.3033 \times 0.2840) + (0.1684 \times 0.2860) + (0.1576 \times 0.1680) + (-0.0222 \times -0.2020) \\
&\quad + (0.0176 \times -0.2740) + (0.4436 \times 0.1720) + (-0.1007 \times 0.0600) + (-0.0965 \times 0.0170) = 0.2291 \\
FFN[3] &= (0.3033 \times 0.1290) + (0.1684 \times -0.2960) + (0.1576 \times -0.1810) + (-0.0222 \times 0.0580) \\
&\quad + (0.0176 \times 0.2740) + (0.4436 \times 0.2200) + (-0.1007 \times -0.1990) + (-0.0965 \times 0.2630) = 0.0565
\end{aligned}$$

Resulting FFN matrix (shape 4×4):

<bos>	-0.0236	-0.0162	-0.1371	0.0116
I	0.0244	0.0713	0.0595	0.1548
love	-0.0430	-0.1132	-0.0617	-0.0451
AI	0.0610	0.1164	0.2291	0.0565

Step 14: Residual add + final LayerNorm -> LAYER OUTPUT

Yn (shape 4×4):

<bos>	-0.7108	1.5825	0.1140	-0.9857
I	1.6133	-0.5046	-1.0672	-0.0414
love	-1.4237	1.4002	0.0864	-0.0629
AI	0.9900	-1.6500	0.1314	0.5286

FFN_out (shape 4×4):

<bos>	-0.0236	-0.0162	-0.1371	0.0116
I	0.0244	0.0713	0.0595	0.1548
love	-0.0430	-0.1132	-0.0617	-0.0451
AI	0.0610	0.1164	0.2291	0.0565

Input matrix (every token as a row) (shape 4×4):

<bos>	-0.7344	1.5664	-0.0231	-0.9742
I	1.6376	-0.4333	-1.0077	0.1134
love	-1.4667	1.2870	0.0247	-0.1080
AI	1.0510	-1.5336	0.3605	0.5852

$\gamma = 1$, $\beta = 0$ (identity affine, to isolate the normalization math)

<bos>: $v = [-0.7344, 1.5664, -0.0231, -0.9742]$

$$\text{mean} = \frac{-0.7344 + 1.5664 + -0.0231 + -0.9742}{4} = -0.0413$$

$$\text{deviations} = [-0.6931, 1.6077, 0.0182, -0.9328] \Rightarrow \text{variance} = \frac{3.9355}{4} = 0.9839$$

$$\text{std} = \sqrt{0.9839 + 10^{-5}} = 0.9919$$

$$\text{normalized} = [\frac{-0.6931}{0.9919} = -0.6987, \frac{1.6077}{0.9919} = 1.6208, \frac{0.0182}{0.9919} = 0.0183, \frac{-0.9328}{0.9919} = -0.9404]$$

$$I: v = [1.6376, -0.4333, -1.0077, 0.1134]$$

$$\text{mean} = \frac{1.6376 + -0.4333 + -1.0077 + 0.1134}{4} = 0.0775$$

$$\text{deviations} = [1.5601, -0.5108, -1.0852, 0.0359] \Rightarrow \text{variance} = \frac{3.8739}{4} = 0.9685$$

$$\text{std} = \sqrt{0.9685 + 10^{-5}} = 0.9841$$

$$\text{normalized} = [\frac{1.5601}{0.9841} = 1.5853, \frac{-0.5108}{0.9841} = -0.5190, \frac{-1.0852}{0.9841} = -1.1027, \frac{0.0359}{0.9841} = 0.0364]$$

$$\text{love: } v = [-1.4667, 1.2870, 0.0247, -0.1080]$$

$$\text{mean} = \frac{-1.4667 + 1.2870 + 0.0247 + -0.1080}{4} = -0.0657$$

$$\text{deviations} = [-1.4010, 1.3528, 0.0904, -0.0422] \Rightarrow \text{variance} = \frac{3.8027}{4} = 0.9507$$

$$\text{std} = \sqrt{0.9507 + 10^{-5}} = 0.9750$$

$$\text{normalized} = [\frac{-1.4010}{0.9750} = -1.4368, \frac{1.3528}{0.9750} = 1.3874, \frac{0.0904}{0.9750} = 0.0927, \frac{-0.0422}{0.9750} = -0.0433]$$

$$AI: v = [1.0510, -1.5336, 0.3605, 0.5852]$$

$$\text{mean} = \frac{1.0510 + -1.5336 + 0.3605 + 0.5852}{4} = 0.1157$$

$$\text{deviations} = [0.9353, -1.6494, 0.2447, 0.4694] \Rightarrow \text{variance} = \frac{3.8754}{4} = 0.9689$$

$$\text{std} = \sqrt{0.9689 + 10^{-5}} = 0.9843$$

$$\text{normalized} = [\frac{0.9353}{0.9843} = 0.9502, \frac{-1.6494}{0.9843} = -1.6757, \frac{0.2447}{0.9843} = 0.2486, \frac{0.4694}{0.9843} = 0.4769]$$

Resulting normalized matrix (shape 4×4):

<bos>	-0.6987	1.6208	0.0183	-0.9404
I	1.5853	-0.5190	-1.1027	0.0364
love	-1.4368	1.3874	0.0927	-0.0433
AI	0.9502	-1.6757	0.2486	0.4769

Step 15: Next-token prediction — how a hidden vector becomes a word

The same embedding table is used at both ends of the network (weight tying). A word's embedding and a position's final hidden state live in the same vector space, so predicting a word is really asking: which word's own embedding vector does this hidden state point most closely toward? That similarity contest is a dot product against every candidate word's embedding, followed by softmax to turn the resulting alignment scores into probabilities.

Emb (tied embedding/output matrix) (shape 4×4):

a	0.1500	0.0500	-0.0500	0.1000
lot	0.0500	-0.1500	0.2000	0.0500
AI	0.3000	-0.2000	0.1000	0.2000
love	-0.1000	0.3000	0.2000	0.1000

Emb^T (shape 4×4):

$$\begin{bmatrix} 0.1500 & 0.0500 & 0.3000 & -0.1000 \\ 0.0500 & -0.1500 & -0.2000 & 0.3000 \\ -0.0500 & 0.2000 & 0.1000 & 0.2000 \\ 0.1000 & 0.0500 & 0.2000 & 0.1000 \end{bmatrix}$$

$final_output$ (every token as a row) (shape 4×4):

<bos>	-0.6987	1.6208	0.0183	-0.9404
I	1.5853	-0.5190	-1.1027	0.0364
love	-1.4368	1.3874	0.0927	-0.0433
AI	0.9502	-1.6757	0.2486	0.4769

Expanded arithmetic, $logit = final_output \cdot Emb^T$, every token:

<bos> ($final_output$ row = $[-0.6987, 1.6208, 0.0183, -0.9404]$)

$$logit[0] = (-0.6987 \times 0.1500) + (1.6208 \times 0.0500) + (0.0183 \times -0.0500) + (-0.9404 \times 0.1000) = -0.1187$$

$$logit[1] = (-0.6987 \times 0.0500) + (1.6208 \times -0.1500) + (0.0183 \times 0.2000) + (-0.9404 \times 0.0500) = -0.3214$$

$$logit[2] = (-0.6987 \times 0.3000) + (1.6208 \times -0.2000) + (0.0183 \times 0.1000) + (-0.9404 \times 0.2000) = -0.7200$$

$$logit[3] = (-0.6987 \times -0.1000) + (1.6208 \times 0.3000) + (0.0183 \times 0.2000) + (-0.9404 \times 0.1000) = 0.4657$$

I ($final_output$ row = $[1.5853, -0.5190, -1.1027, 0.0364]$)

$$logit[0] = (1.5853 \times 0.1500) + (-0.5190 \times 0.0500) + (-1.1027 \times -0.0500) + (0.0364 \times 0.1000) = 0.2706$$

$$logit[1] = (1.5853 \times 0.0500) + (-0.5190 \times -0.1500) + (-1.1027 \times 0.2000) + (0.0364 \times 0.0500) = -0.0616$$

$$logit[2] = (1.5853 \times 0.3000) + (-0.5190 \times -0.2000) + (-1.1027 \times 0.1000) + (0.0364 \times 0.2000) = 0.4764$$

$$logit[3] = (1.5853 \times -0.1000) + (-0.5190 \times 0.3000) + (-1.1027 \times 0.2000) + (0.0364 \times 0.1000) = -0.5311$$

love ($final_output$ row = $[-1.4368, 1.3874, 0.0927, -0.0433]$)

$$logit[0] = (-1.4368 \times 0.1500) + (1.3874 \times 0.0500) + (0.0927 \times -0.0500) + (-0.0433 \times 0.1000) = -0.1551$$

$$logit[1] = (-1.4368 \times 0.0500) + (1.3874 \times -0.1500) + (0.0927 \times 0.2000) + (-0.0433 \times 0.0500) = -0.2636$$

$$logit[2] = (-1.4368 \times 0.3000) + (1.3874 \times -0.2000) + (0.0927 \times 0.1000) + (-0.0433 \times 0.2000) = -0.7079$$

$$logit[3] = (-1.4368 \times -0.1000) + (1.3874 \times 0.3000) + (0.0927 \times 0.2000) + (-0.0433 \times 0.1000) = 0.5741$$

AI ($final_output$ row = $[0.9502, -1.6757, 0.2486, 0.4769]$)

$$\text{logit}[0] = (0.9502 \times 0.1500) + (-1.6757 \times 0.0500) + (0.2486 \times -0.0500) + (0.4769 \times 0.1000) = 0.0940$$

$$\text{logit}[1] = (0.9502 \times 0.0500) + (-1.6757 \times -0.1500) + (0.2486 \times 0.2000) + (0.4769 \times 0.0500) = 0.3724$$

$$\text{logit}[2] = (0.9502 \times 0.3000) + (-1.6757 \times -0.2000) + (0.2486 \times 0.1000) + (0.4769 \times 0.2000) = 0.7404$$

$$\text{logit}[3] = (0.9502 \times -0.1000) + (-1.6757 \times 0.3000) + (0.2486 \times 0.2000) + (0.4769 \times 0.1000) = -0.5003$$

Resulting logit matrix (shape 4×4):

	<i>a</i>	<i>lot</i>	<i>AI</i>	<i>love</i>
<bos>	-0.1187	-0.3214	-0.7200	0.4657
I	0.2706	-0.0616	0.4764	-0.5311
love	-0.1551	-0.2636	-0.7079	0.5741
AI	0.0940	0.3724	0.7404	-0.5003

Step 15b: Softmax over the vocabulary, every position, fully explained

Five steps turn a row of logits into a genuine probability distribution: (1) find the max, (2) subtract it from every logit — a numerical-stability trick that changes nothing mathematically, since subtracting a constant before exponentiating multiplies numerator and denominator by the same factor, which cancels, (3) exponentiate every shifted value, (4) sum the exponentials, (5) divide each exponential by that sum.

Position “<bos>”

$$\text{logits} = [\text{a} = -0.1187, \text{lot} = -0.3214, \text{AI} = -0.7200, \text{love} = 0.4657]$$

$$\text{max} = 0.4657$$

$$\text{shifted} = [-0.5845, -0.7871, -1.1858, 0.0000]$$

$$\text{exponentiate} = [e^{-0.5845} = 0.5574, e^{-0.7871} = 0.4551, e^{-1.1858} = 0.3055, e^{0.0000} = 1.0000] \quad \text{sum} = 2.3181$$

$$\text{probabilities} = [\frac{0.5574}{2.3181} = 0.2405, \frac{0.4551}{2.3181} = 0.1963, \frac{0.3055}{2.3181} = 0.1318, \frac{1.0000}{2.3181} = 0.4314]$$

predicted next token after “<bos>”: love (probability 0.4314)

Position “I”

$$\text{logits} = [\text{a} = 0.2706, \text{lot} = -0.0616, \text{AI} = 0.4764, \text{love} = -0.5311]$$

$$\text{max} = 0.4764$$

$$\text{shifted} = [-0.2058, -0.5380, 0.0000, -1.0076]$$

$$\text{exponentiate} = [e^{-0.2058} = 0.8140, e^{-0.5380} = 0.5839, e^{0.0000} = 1.0000, e^{-1.0076} = 0.3651] \quad \text{sum} = 2.7630$$

$$\text{probabilities} = [\frac{0.8140}{2.7630} = 0.2946, \frac{0.5839}{2.7630} = 0.2113, \frac{1.0000}{2.7630} = 0.3619, \frac{0.3651}{2.7630} = 0.1321]$$

predicted next token after “I”: AI (probability 0.3619)

Position “love”

$$\text{logits} = [\text{a} = -0.1551, \text{lot} = -0.2636, \text{AI} = -0.7079, \text{love} = 0.5741]$$

$$\text{max} = 0.5741$$

$$\text{shifted} = [-0.7292, -0.8377, -1.2821, 0.0000]$$

$$\text{exponentiate} = [e^{-0.7292} = 0.4823, e^{-0.8377} = 0.4327, e^{-1.2821} = 0.2775, e^{0.0000} = 1.0000] \quad \text{sum} = 2.1924$$

$$\text{probabilities} = [\frac{0.4823}{2.1924} = 0.2200, \frac{0.4327}{2.1924} = 0.1974, \frac{0.2775}{2.1924} = 0.1266, \frac{1.0000}{2.1924} = 0.4561]$$

predicted next token after “love”: love (probability 0.4561)

Position “AI”

$$\text{logits} = [\text{a} = 0.0940, \text{lot} = 0.3724, \text{AI} = 0.7404, \text{love} = -0.5003]$$

$$\text{max} = 0.7404$$

$$\text{shifted} = [-0.6464, -0.3680, 0.0000, -1.2407]$$

$$\text{exponentiate} = [e^{-0.6464} = 0.5239, e^{-0.3680} = 0.6921, e^{0.0000} = 1.0000, e^{-1.2407} = 0.2892] \quad \text{sum} = 2.5052$$

$$\text{probabilities} = [\frac{0.5239}{2.5052} = 0.2091, \frac{0.6921}{2.5052} = 0.2763, \frac{1.0000}{2.5052} = 0.3992, \frac{0.2892}{2.5052} = 0.1154]$$

predicted next token after “AI”: AI (probability 0.3992)

Step 16: Cross-entropy loss — the actual training signal, every position

This is the step that makes training possible: at each position, compare the predicted probability distribution against the token that ACTUALLY came next in the real training text, and penalize the model in proportion to how much probability it failed to assign to the correct answer. This scalar loss is what gradient descent minimizes – its gradient flows backward through every step in this document and nudges every weight matrix shown so far a small amount in the direction that would have made the correct token more likely.

$$\mathcal{L} = -\log(P(\text{true_next_token}))$$

Position “<bos>” (true next token = “AI”)

$$P(\text{AI}) = 0.1318 \quad \Rightarrow \quad \mathcal{L} = -\log(0.1318) = 2.0265$$

Position “I” (true next token = “love”)

$$P(\text{love}) = 0.1321 \quad \Rightarrow \quad \mathcal{L} = -\log(0.1321) = 2.0239$$

Position “love” (true next token = “AI”)

$$P(\text{AI}) = 0.1266 \quad \Rightarrow \quad \mathcal{L} = -\log(0.1266) = 2.0671$$

Position “AI” (true next token = “a”)

$$P(\text{a}) = 0.2091 \quad \Rightarrow \quad \mathcal{L} = -\log(0.2091) = 1.5648$$

$$\text{Average loss} = \frac{2.0265 + 2.0239 + 2.0671 + 1.5648}{4} = \frac{7.6822}{4} = 1.9206$$

This single number is what one gradient-descent step tries to reduce. Notice the loss is LOWEST wherever the model’s (currently untrained, hand-picked) weights happened to already assign high probability to the true answer, and HIGHEST wherever they didn’t – training is nothing more than repeating this exact forward pass, computing this exact loss, and adjusting every weight matrix in this document a tiny amount in the direction that reduces it, over and over, across billions of real sentences.

decoder-only LLM walkthrough in code -- token + positional embeddings, LayerNorm, standard multi-head causal attention, a dense feed-forward network, next-token prediction, and the cross-entropy training loss -- with hands-on exercises at the end for extending the example.

A Simple (Vanilla GPT-Style) Decoder LLM — Full Worked Example: " I love AI"

Replicates every calculation from the walkthrough in executable code: token + learned positional embeddings, pre-norm LayerNorm, standard multi-head causal self-attention (no compression, no rotation), a standard dense feed-forward network (no experts), next-token prediction, and the cross-entropy training loss.

In [1]:

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

tokens = ["<bos>", "I", "love", "AI"]
n = len(tokens)
d_model, num_heads, d_k = 4, 2, 2
print(f"Tokens: {tokens}")
```

Tokens: ['<bos>', 'I', 'love', 'AI']

Steps 2-4: Token embedding + learned positional embedding

In [2]:

```
token_emb = {
    "<bos>": np.array([0.05,0.05,0.05,0.05]),
    "I": np.array([0.20,0.10,-0.10,0.00]),
    "love": np.array([-0.10,0.30,0.20,0.10]),
    "AI": np.array([0.30,-0.20,0.10,0.20]),
    "a": np.array([0.15,0.05,-0.05,0.10]),
}
X_tok = np.array([token_emb[t] for t in tokens])

pos_table = np.array([
    [0.00, 0.10, 0.00, 0.00],
    [0.10, 0.00, 0.10, 0.05],
    [0.00, 0.10, -0.05, 0.10],
    [0.05, -0.05, 0.10, 0.00],
])

X = X_tok + pos_table
print("X (token + position):\n", X)
```

```
X (token + position):
[[ 0.05  0.15  0.05  0.05]
 [ 0.3   0.1   0.   0.05]
 [-0.1   0.4   0.15  0.2 ]
 [ 0.35 -0.25  0.2   0.2 ]]
```

Step 5: Pre-attention LayerNorm (standard, mean-centered)

In [3]:

```
def layernorm(v, gamma=1.0, beta=0.0, eps=1e-5):
    mean = v.mean(axis=-1, keepdims=True)
    var = v.var(axis=-1, keepdims=True)
    return gamma*(v-mean)/np.sqrt(var+eps) + beta

Xn = layernorm(X)
print("Xn:\n", Xn)
```

```
Xn:
[[-0.5758  1.7275 -0.5758 -0.5758]
 [ 1.6458 -0.1097 -0.9875 -0.5486]
 [-1.4737  1.3333 -0.0702  0.2105]
 [ 0.9999 -1.6665  0.3333  0.3333]]
```

Steps 6-7: Standard multi-head causal self-attention — no compression, no rotation

In [4]:

```
Wq = [
    np.array([[1.0,0.0],[0.0,1.0],[0.5,0.0],[0.0,0.5]]),
    np.array([[0.0,1.0],[1.0,0.0],[0.0,0.5],[0.5,0.0]]),
]
Wk = [
    np.array([[0.8,0.0],[0.0,0.8],[0.3,0.0],[0.0,0.3]]),
    np.array([[0.0,0.8],[0.8,0.0],[0.0,0.3],[0.3,0.0]]),
]
Wv = [
    np.array([[0.2,0.1],[0.1,0.2],[0.0,0.3],[0.3,0.0]]),
    np.array([[0.1,0.2],[0.2,0.1],[0.3,0.0],[0.0,0.3]]),
]

def softmax(x, axis=-1):
    e = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e/e.sum(axis=axis, keepdims=True)

causal_mask = np.triu(np.ones((n,n)), k=1).astype(bool)
print("Causal mask (True = blocked):\n", causal_mask)

head_outputs = []
for h in range(num_heads):
    Q, K, V = Xn @ Wq[h], Xn @ Wk[h], Xn @ Wv[h]
    scores = (Q @ K.T)/np.sqrt(d_k)
    scores_masked = np.where(causal_mask, -1e9, scores)
    weights = softmax(scores_masked, axis=-1)
    out = weights @ V
    head_outputs.append(out)
    print(f"--- Head {h+1} ---")
    print("softmax weights:\n", weights)
    print("head output:\n", out)
```

```
Causal mask (True = blocked):
[[False True  True  True]
 [False False True  True]
 [False False False True]
 [False False False False]]
--- Head 1 ---
softmax weights:
[[1.    0.    0.    0.   ]
 [0.1488 0.8512 0.    0.   ]
 [0.3667 0.0142 0.6191 0.   ]
 [0.0148 0.2737 0.0101 0.7013]]
head output:
[[-0.1152  0.1152]
 [ 0.1136 -0.1136]
 [-0.1009  0.1009]
 [ 0.1328 -0.1328]]
--- Head 2 ---
softmax weights:
[[1.    0.    0.    0.   ]
 [0.1488 0.8512 0.    0.   ]
 [0.3667 0.0142 0.6191 0.   ]
 [0.0148 0.2737 0.0101 0.7013]]
head output:
[[ 0.1152 -0.1152]
 [-0.1136  0.1136]
 [ 0.1009 -0.1009]
 [-0.1328  0.1328]]
```

Steps 8-10: Concat heads, project through Wo, residual, second LayerNorm

In [5]:

```
concat = np.concatenate([head_outputs[0], head_outputs[1]], axis=1)
Wo = np.array([[0.4,0.1,0.2,0.3],[0.2,0.3,0.1,0.0],[0.1,0.2,0.4,0.1],[0.3,0.0,0.1,0.4]])
attn_out = concat @ Wo
Y = X + attn_out # residual adds back to ORIGINAL X (pre-norm architecture)
Yn = layernorm(Y)
print("MultiHead(X):\n", attn_out)
print("Y = X + MultiHead(X):\n", Y)
print("Yn = LayerNorm(Y):\n", Yn)
```

```
MultiHead(X):
[[-0.0461  0.0461  0.023  -0.0691]
 [ 0.0454 -0.0454 -0.0227  0.0682]
 [-0.0403  0.0403  0.0202 -0.0605]
 [ 0.0531 -0.0531 -0.0266  0.0797]]
Y = X + MultiHead(X):
[[ 0.0039  0.1961  0.073  -0.0191]
 [ 0.3454  0.0546 -0.0227  0.1182]
 [-0.1403  0.4403  0.1702  0.1395]
 [ 0.4031 -0.3031  0.1734  0.2797]]
Yn = LayerNorm(Y):
[[-0.7108  1.5825  0.114  -0.9857]
 [ 1.6133 -0.5046 -1.0672 -0.0414]
 [-1.4237  1.4002  0.0864 -0.0629]
 [ 0.99   -1.65   0.1314  0.5286]]
```

Steps 11-14: Standard dense feed-forward network (no experts, no routing)

In [6]:

```
def gelu(x): return 0.5*x*(1+np.tanh(np.sqrt(2/np.pi)*(x+0.044715*x**3)))

W1 = np.round(np.random.RandomState(3).uniform(-0.3,0.3,size=(4,8)),3)
W2 = np.round(np.random.RandomState(4).uniform(-0.3,0.3,size=(8,4)),3)

hidden = Yn @ W1
activated = gelu(hidden)
ffn_out = activated @ W2

residual2 = Yn + ffn_out
final_output = layernorm(residual2)
print("FFN output:\n", ffn_out)
print("Final layer output (after residual + LayerNorm):\n", final_output)
```

```
FFN output:
[[-0.0236 -0.0162 -0.1371  0.0116]
 [ 0.0244  0.0713  0.0595  0.1548]
 [-0.043  -0.1132 -0.0617 -0.0451]
 [ 0.061   0.1164  0.2291  0.0565]]
Final layer output (after residual + LayerNorm):
[[-0.6987  1.6208  0.0183 -0.9404]
 [ 1.5853 -0.519  -1.1027  0.0364]
 [-1.4368  1.3874  0.0927 -0.0433]
 [ 0.9502 -1.6757  0.2486  0.4769]]
```

Step 15: Next-token prediction, every position

In [7]:

```
vocab = {"a": np.array([0.15,0.05,-0.05,0.10]), "lot": np.array([0.05,-0.15,0.20,0.05]),
        "AI": np.array([0.30,-0.20,0.10,0.20]), "love": np.array([-0.10,0.30,0.20,0.10])}
vocab_list = list(vocab.keys())
Emb = np.array([vocab[w] for w in vocab_list])

logits_all = final_output @ Emb.T
probs_all = []
for i,t in enumerate(tokens):
    row = logits_all[i]
    probs = np.exp(row-row.max()); probs /= probs.sum()
    probs_all.append(probs)
    pred = vocab_list[np.argmax(probs)]
    print(f"After '{t}': probs={dict(zip(vocab_list, np.round(probs,4)))} -> predicted '{pred}'")
probs_all = np.array(probs_all)
```

```
After '<bos>': probs={'a': np.float64(0.2405), 'lot': np.float64(0.1963), 'AI': np.float64(0.1318), 'love': np.float64(0.4314)} -> predicted 'love'
After 'I': probs={'a': np.float64(0.2946), 'lot': np.float64(0.2113), 'AI': np.float64(0.3619), 'love': np.float64(0.1321)} -> predicted 'AI'
After 'love': probs={'a': np.float64(0.22), 'lot': np.float64(0.1974), 'AI': np.float64(0.1266), 'love': np.float64(0.4561)} -> predicted 'love'
After 'AI': probs={'a': np.float64(0.2091), 'lot': np.float64(0.2763), 'AI': np.float64(0.3992), 'love': np.float64(0.1154)} -> predicted 'AI'
```

Step 16: Cross-entropy loss — the actual training signal

Compare each position's predicted distribution against the token that truly came next, and take the negative log of the probability assigned to it.

In [8]:

```
true_targets = {"<bos>": "AI", "I": "love", "love": "AI", "AI": "a"}

losses = []
for i,t in enumerate(tokens):
    true_tok = true_targets[t]
    true_idx = vocab_list.index(true_tok)
    p_true = probs_all[i, true_idx]
    loss = -np.log(p_true)
    losses.append(loss)
    print(f"Position '{t}' (true next = '{true_tok}'): P={p_true:.4f}, loss=-log({p_true:.4f})={loss:.4f}")

print(f"\nAverage loss = {np.mean(losses):.4f}")
print("This is the scalar that gradient descent minimizes -- its gradient flows back through")
print("every weight matrix used above (W1/W2, Wo, Wq/Wk/Wv, both embedding tables).")
```

```
Position '<bos>' (true next = 'AI'): P=0.1318, loss=-log(0.1318)=2.0265
Position 'I' (true next = 'love'): P=0.1321, loss=-log(0.1321)=2.0239
Position 'love' (true next = 'AI'): P=0.1266, loss=-log(0.1266)=2.0671
Position 'AI' (true next = 'a'): P=0.2091, loss=-log(0.2091)=1.5648
```

Average loss = 1.9206

This is the scalar that gradient descent minimizes -- its gradient flows back through every weight matrix used above (W1/W2, Wo, Wq/Wk/Wv, both embedding tables).

Exercise: try it yourself

1. **Compare against DeepSeek's version:** swap this attention block for the MLA version from the DeepSeek notebook and confirm the causal masking / softmax mechanics are identical, only the Q/K/V construction differs.
2. **Verify weight tying:** try using a *separate*, untied output matrix instead of reusing the input embedding table, and see how the parameter count changes.
3. **Add a second layer:** feed `final_output` back in as the new `X` and re-run Steps 5-14 (skip the embedding lookups -- they only happen once).
4. **Try a different loss:** replace one `true_targets` entry with a token the model assigned very low probability to, and confirm the loss for that position spikes accordingly.
5. **Implement label smoothing:** instead of the true target having probability 1.0 in the cross-entropy target, spread a small amount (e.g. 0.1) of probability mass across the other vocabulary words before computing the loss, and see how it changes the gradient signal.

In [9]:

```
# Your experiments here
```